

ESERCITAZIONE 3

# Corso di
# CVeDI (8 CFU)

**Elia Guarnieri**
***elia.guarnieri@unimib.it***

# Nuove proprietà CSS, Grid e Srcset

1. Proprietà Css 2025
2. Grid
3. Srcset

# PROPRIETÀ CSS DA CONOSCERE NEL 2025

# Proprietà CSS, compatibilità e prefissi

Periodicamente, vengono introdotte nuove proprietà CSS che non fanno parte fin da subito delle specifiche supportate da tutti i browser

I prefissi delle proprietà CSS, come "-webkit-", "-moz-" servono a coloro che sviluppano i browser (conosciuti come vendors) per **aggiungere nuove proprietà CSS in modo sicuro**, riservandosi una finestra temporale nella quale queste proprietà sono "sperimentali", in fase di test

Tutte queste proprietà faranno parte, prima o poi, delle specifiche. Utilizzare i prefissi permette allo sviluppatore front-end di avere un "accesso anticipato" a esse, garantendo al contempo la compatibilità

CanIUse - per controllare la compatibilità

# Proprietà da tenere d’occhio

**Container Queries**
https://developer.mozilla.org/en-US/docs/container_queries

**Dynamic Viewport Units**
https://developer.mozilla.org/en-US/docs/CSS/Vlues_and_units

**Logical Properties (Inline and Block)**
https://developer.mozilla.org/en-US/docs/logical_properties

**CSS nesting**
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_nesting

# Container Queries

Per approfondire

Le "container queries" ti permettono di applicare stili a un elemento in base alla dimensione del contenitore dell'elemento

Ad esempio, se un contenitore ha meno spazio disponibile nel contesto circostante, puoi nascondere alcuni elementi o utilizzare caratteri più piccoli

Le "container queries" sono una alternativa alle "media queries", che applicano stili agli elementi in base alla dimensione del viewport o ad altre caratteristiche del dispositivo

```
/* If the container is larger than 700px */
@container (min-width: 700px) {
 .card h2 {
   font-size: 2em;
 }
}
```

# Dynamic Viewport Units

Per approfondire

Le Viewport Unit su mobile non tengono conto della barra di navigazione o di un eventuale bottom menu. Questo causa alcune problematiche di visualizzazione quando lo stato della pagina si aggiorna (e.g., scrollando la barra di navigazione si nasconde)

Sono quindi state introdotte delle nuove Viewport Units che tengono conto di questi elementi: small, large, dynamic viewport units

```
element {
 property: 10dvh;
 property: 10lvh;
 property: 10svh;
}
```

# Logical Properties

Per approfondire

Le proprietà e i valori logici utilizzano i termini astratti "block" e "inline" per descrivere la direzione in cui fluiscono. Il significato fisico di questi termini dipende dalla modalità di scrittura

Lo stesso Bootstrap ha introdotto nelle ultime versione le logical properties (cfr. Bootstrap spacing)

```
element {
 margin-block-start: 1rem;
 margin-block-end: 2rem;
 margin-inline: 3rem;
}
```

# CSS Nesting

[Per approfondire]

Il modulo di nidificazione CSS definisce una sintassi per la nidificazione dei selettori, offrendo la possibilità di annidare una regola di stile all'interno di un'altra. La nidificazione CSS è diversa dai preprocessori CSS come Sass nel senso che viene analizzata dal browser anziché essere precompilata da un preprocessore CSS

La nidificazione CSS aiuta nella leggibilità, modularità e manutenibilità dei fogli di stile CSS. Inoltre, potenzialmente aiuta a ridurre le dimensioni dei file CSS, riducendo così la quantità di dati scaricati dagli utenti

```
parentRule {
 /* parent rule style properties */
 & childRule {
   /* child rule style properties */
 }
}
```

=

```
.parent-rule {
 /* parent rule style properties */
}
.parent-rule .child-rule {
 /* style properties for .child-rule
descendants for .parent-rule ancestors */
}
```

# Proprietà generalmente supportate

**Object-fit**
https://developer.mozilla.org/en-US/docs/Web/CSS/object-fit

**Clip-path**
https://developer.mozilla.org/en-US/docs/Web/CSS/clip-path

**Aspect-ratio**
https://developer.mozilla.org/en-US/docs/Web/CSS/aspect-ratio

**Columns**
https://developer.mozilla.org/en-US/docs/Web/CSS/columns

# Object-fit

Try it

La proprietà object-fit definisce il modo in cui un elemento si adatta all'altezza e alla larghezza del box che lo contiene

Si utilizza su elementi come **immagini e video**, e permette di acquisire il controllo su come l'elemento si "schiaccia" o si "allarga" nel suo contenitore

Per modificare l'allineamento dell'elemento ridimensionato rispetto al suo contenitore si può usare la proprietà **object-position**

Per approfondire la differenza tra background-image e object -fit

- A Deep Dive Into object-fit And background-size In CSS

```
img {
  height: 200px;
  width: 200px;
  object-fit: fill | contain | cover | none | scale-down
}
```

# Clip-path

Try it

clip-path permette di isolare nell'elemento la specifica regione che deve essere mostrata. Uno dei suoi utilizzi più semplici prevede la creazione di una regione sfruttando **semplici forme geometriche** (circle, ellipses, polygon)

Si può altrimenti utilizzare un SVG contenente un path

Un sito utile per creare particolari forme

- Clippy — CSS clip-path maker

```
/*
 * Per mostrare soltanto una regione circolare di 60px
 * di diametro posizionata al centro dell'immagine
 */
img {
  clip-path: circle(60px at center);
}
```

# Aspect-ratio

Try it

La proprietà CSS aspect-ratio imposta un **rapporto di aspetto preferito per il box**, che sarà utilizzato **nel calcolo delle dimensioni automatiche** e in alcune altre funzioni di layout

Se viene definita una dimensione, l'altra verrà calcolata in automatico in base al rapporto d'aspetto selezionato

Può essere un valido strumento da abbinare a object-fit nella creazione di immagini responsive

Per approfondire

- esempio
- Aspect Ratio is Great

```
img {
  aspect-ratio: 1 / 1; /* img sarà quadrato */
}
```

# Columns

Try it

Grazie a columns potete creare un layout ispirato alla stampa, dotato tuttavia della flessibilità necessaria sul web.

La proprietà accetta i valori **column-count** (intero) o **column-width** (dimensione), o entrambe

Si può abbinare a **column-gap**, che indica lo spazio tra le colonne

Le colonne sono già responsive e non c'è bisogno di utilizzare media queries per cambiare il numero di colonne. Il browser userà la larghezza ottimale da noi specificata per decidere quando spostare una colonna sotto l'altra

```
.testo {
  columns: 300px 3; /* crea tre colonne di 300px ciascuna */
  column-gap: 10px; /* crea un gap di 10px tra ogni colonna */
}
```

# GRID

# Cos’è grid

Il grid layout di CSS è un potente strumento per creare layout

Al contrario di flexbox, che si basa su una sola dimensione, grid è “bidimensionale”, permettendo di gestire righe e colonne

Le regole si possono applicare a un elemento genitore (il **contenitore grid**) e agli elementi figli (i **grid items**)

Reference

# Proprietà per il genitore

Per dichiarare un contenitore grid basta assegnargli **display: grid;**

Al genitore possono essere dunque assegnate diverse proprietà, tra cui:

- grid-template-columns e grid-template-rows
- grid-template-areas
- column-gap e row-gap

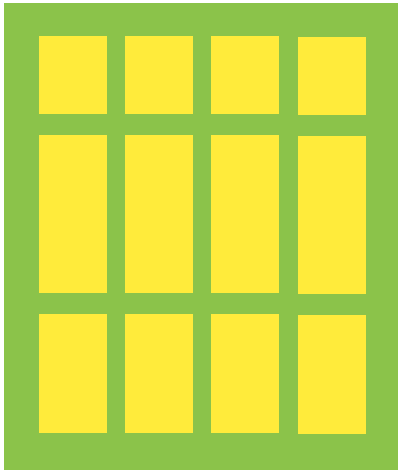
# grid-template-columns e grid-template-row

Queste proprietà definiscono le righe e le colonne della vostra griglia, grazie ad una lista di valori separati da uno spazio

Ogni valore rappresenta le dimensioni della vostra colonna o della vostra riga

Gli spazi tra colonne e righe sono detti linee

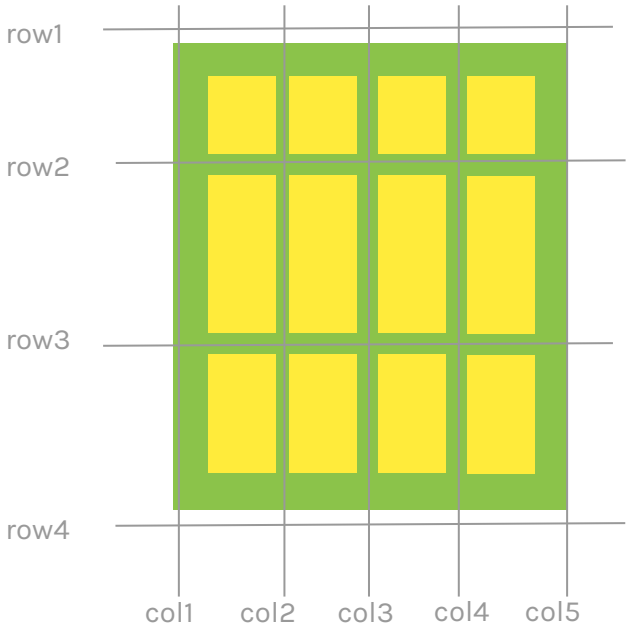
```
.container {
  grid-template-columns: auto auto auto auto;
  grid-template-rows: 100px 250px 150px;
}
```



# grid-template-columns e grid-template-row

Potete assegnare un nome alle linee usando parentesi quadre

Questo vi aiuterà a posizionare i grid items




```
.container {
  grid-template-columns: [col1] auto [col2] auto [col3] auto [col4] auto [col5];
  grid-template-rows: [row1] 100px [row2] 250px [row3] 150px [row4];
}
```

# grid-template-areas

Questa proprietà permette di definire dei nomi arbitrari per le aree della vostra griglia

La sintassi stessa permette di “visualizzare” la struttura della griglia

Questo permetterà di posizionare più comodamente i grid items (abbinandolo con grid-area)

O addirittura creare un layout completamente responsive senza l’utilizzo delle media queries

- Esempio interattivo
- Look Ma, No Media Queries! Responsive Layouts Using CSS Grid

```
.container {
  grid-template-columns: auto auto auto
auto;
  grid-template-rows: 100px 250px 150px;
  grid-template-areas:
    "header header header header"
    "main main main side"
    "footer footer footer side";
}

.header {
  grid-area: header;
}
.main {
  grid-area: main;
}
```

# column-gap e row-gap

Permette di definire quello che potete immaginare come il gutter tra colonne o tra righe

Entrambe le proprietà accettano una dimensione come valore

```
.container {
  column-gap: <line-size>;
  row-gap: <line-size>;
}
```

# Proprietà per i figli

- grid-column e grid-row
- grid-area

# grid-column e grid-row

Sono rispettivamente forme contratte di 4 altre proprietà:
**grid-column-start** + **grid-column-end** e **grid-row-start** + **grid-row-end**

I valori accettati sono il numero (o il nome) della linea da cui deve partire l'elemento, unito a un "/" e al numero (o al nome) della linea di fine

Alternativamente, il secondo argomento può essere "span" seguito da un numero che indicherà quante righe/colonne dovrà occupare l'elemento

```
.item {
  grid-column: <start-line> / <end-line> | <start-line> / span <value>;
  grid-row: <start-line> / <end-line> | <start-line> / span <value>;
}
```

# grid-area

Dovete abbinarlo a grid-template-areas per permettere di posizionare direttamente un elemento su una determinata area della griglia.

srcset

# Cos’è srcset e perché usarlo

In Html è possibile servire all’utente un’immagine differente in base alle circostanze

Usando i **srcset** si fornisce al browser la possibilità di scegliere di mostrare una versione più leggera dell’immagine, **se lo ritiene**

Il risultato è una potenziale riduzione significativa del tempo di caricamento generale, soprattutto su dispositivi piccoli

https://developer.mozilla.org/en-US/docs/Web/API/HTMLImageElement/srcset

# La sintassi di srcset

Oltre al normale attributo src, si aggiunge srcset per definire versioni della stessa immagine con dimensioni differenti

```
<img srcset="" src="">
```

Il browser assumerà che ogni elemento del srcset è visivamente identico, e sceglierà di utilizzare l'elemento che riterrà più adatto

Non è consigliato inserire immagini diverse

# Implementare srcset con gli x-descriptors

Il modo più semplice di implementare srcset è utilizzare gli **x descriptors**: si tratta di una etichetta apposta ad ogni immagine per schermi a diversa densità

Nel src si fornisce il file di "**default**", di solito a bassa risoluzione, e nel srcset si indicano i file per schermi con una **pixel-density** maggiore

```
<img
  src="image.jpg"
  srcset="image-large.jpg 2x, image-extra-large.jpg 4x"
>
```

# Limitazioni degli x-descriptors

Gli x-descriptors presentano diverse limitazioni perché permettono al browser di basarsi **soltanto sul parametro della pixel-density** del display, e non sulla dimensione dell'immagine nel layout

Visto che le nostre immagini saranno molto spesso inserite nel contesto di un layout responsive, dobbiamo fare in modo che il browser possa decidere anche in base a questo fattore

È per questo che entrano in gioco i **w-descriptors** e l'attributo **sizes**

# Srcset con i w-descriptors

Con l'uso dei w-descriptors stiamo ancora fornendo copie multiple della stessa immagine e lasciando al browser la scelta di quella più appropriata.

Tuttavia, anziché etichettarle con una densità di pixel (x), le stiamo etichettando con la larghezza della risorsa. Quindi, se img-small.jpg è 300×450, lo etichettiamo come 300w e lo stesso faremo con tutte le altre dimensioni

```
<img srcset="image-small.jpg 300w,
             image-medium.jpg 460w,
             image-large.jpg 800w"
>
```

# L’attributo sizes

L'utilizzo di srcset con descrittori di larghezza (w) come questo implica che deve essere abbinato all’attributo sizes in modo che il browser sappia quanto spazio l'immagine occuperà

**Senza queste informazioni, i browser non possono prendere decisioni intelligenti**

La sintassi dell’attributo sizes è

```
<img ... sizes="[media query] [length],
                [media query] [length],
                ...
                [default length]"
>
```

# Implementare srcset con i w-descriptors e sizes

I w-descriptors etichettano le varie versioni delle immagini specificando la loro **larghezza**

Bisogna accoppiare tali descrittori con un attributo sizes, che darà al browser informazioni sullo spazio che l'immagine occuperà all'interno del layout

```
<!— Stiamo prevedendo che l'immagine avrà una larghezza di 280px sugli schermi
fino a 320px, di 440px sugli schermi fino a 480px, di 800px negli altri casi.
Il browser prenderà l'immagine più adeguata dal srcset, scegliendo quella più
vicina per dimensione —>
<img srcset="image-small.jpg 300w,image-medium.jpg 460w, image-large.jpg 800w"
     sizes="(max-width: 320px) 280px, (max-width: 480px) 440px,  800px"
     src="image-small.jpg" alt="An image"
>
```

# Sizes

Quando definite le sizes, ricordate che esse dipenderanno dal vostro layout

In alcuni casi vi sarà possibile conoscere la dimensione precisa dell’immagine sulla pagina

In altri casi, potete fornire una stima che sia “buona abbastanza”, lasciando fare al browser il resto

# Srcset refercence

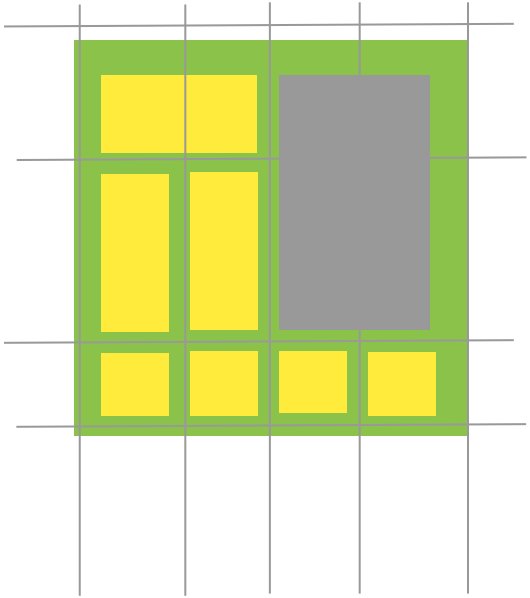
**CSS Tricks**
A Guide to the Responsive Images Syntax in HTML

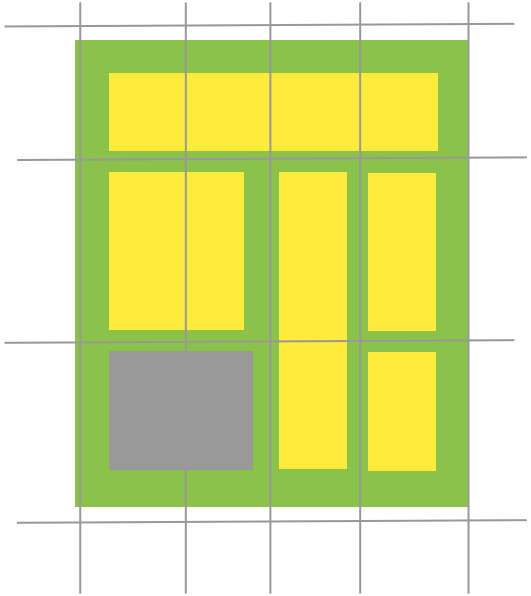
**Smashing Magazine**
Responsive Images Done Right: A Guide To And srcset



# ESERCIZIO 1

- Replicate le grid raffigurate nella slide successiva. Per la prima utilizzate grid-template-areas; per la seconda usate gli attributi dei figli (grid column e grid-row)
- Fate in modo che la grid occupi, in larghezza, il 100% della pagina su mobile, e il 50% su desktop (posizionatela al centro della pagina)
- Nei due quadrati indicati in grigio, inserite un’immagine dotata di attributo srcset con w-descriptors. Create almeno due sizes, cercando di specificarle il più precisamente possibile in base a ciò che conoscete del vostro design







ESERCITAZIONE 3

# FINE